Câu hỏi **1**

Đúng

Đạt điểm 1,00 trên 1,00

Use ANTLR to write regular expressions describing a Pascal identifier that must begin with a lowercase letter ('a' to 'z'), but may continue with many characters which are lowercase letter or digit ('0' to '9').

For example:

Test	Result
`"abc"`	`abc,<EOF>`

Answer: (penalty regime: 10, 20, ... %)

Ace editor not ready. Perhaps reload page?
Falling back to raw text area.

HLang.interp

HLang.tokens

HLangLexer.interp

HLangLexer.py

HLangLexer.tokens

HLangParser.py

HLangVisitor.py

	Test	Expected	Got	
✓	`"abc"`	`abc,<EOF>`	`abc,<EOF>`	✓
✓	`"abc a12"`	`abc,a12,<EOF>`	`abc,a12,<EOF>`	✓
✓	`"abc A12"`	`abc,Error Token A`	`abc,Error Token A`	✓
✓	`"abc?svn"`	`abc,Error Token ?`	`abc,Error Token ?`	✓
✓	`"0a12"`	`Error Token 0`	`Error Token 0`	✓

Câu hỏi **2**

Đúng

Đạt điểm 1,00 trên 1,00

Use ANTLR to write regular expressions describing Pascal tokens For a number to be taken as "real" (or "floating point") format, it must either have a decimal point, or use scientific notation. For example, 1.0, 1e-12, 1.0e-12, 0.000000001 are all valid reals. At least one digit must exist on either side of a decimal point.

For example:

Test	Result
`"1.0"`	`1.0,<EOF>`

Answer: (penalty regime: 10, 20, ... %)

Ace editor not ready. Perhaps reload page?
Falling back to raw text area.

HLang.interp

HLang.tokens

HLangLexer.interp

HLangLexer.py

HLangLexer.tokens

HLangParser.py

HLangVisitor.py

	Test	Expected	Got	
✓	`"1.0"`	`1.0,<EOF>`	`1.0,<EOF>`	✓
✓	`"1e-12"`	`1e-12,<EOF>`	`1e-12,<EOF>`	✓
✓	`"1.0e-12"`	`1.0e-12,<EOF>`	`1.0e-12,<EOF>`	✓
✓	`"0.001"`	`0.001,<EOF>`	`0.001,<EOF>`	✓
✓	`".01"`	`Error Token .`	`Error Token .`	✓

Câu hỏi **3**

Đúng

Đạt điểm 1,00 trên 1,00

Use ANTLR to write regular expressions describing Pascal strings are made up of a sequence of characters between single quotes: 'string'. The single quote itself can appear as two single quotes back to back in a string: 'isn''t'.

For example:

Test	Result
`"'Yanxi Palace - 2018'"`	`'Yanxi Palace - 2018',<EOF>`

Answer: (penalty regime: 10, 20, ... %)

Ace editor not ready. Perhaps reload page?
Falling back to raw text area.

HLang.interp

HLang.tokens

HLangLexer.interp

HLangLexer.py

HLangLexer.tokens

HLangParser.py

HLangVisitor.py

	Test	Expected	Got	
✓	`"'Yanxi Palace - 2018'"`	`'Yanxi Palace - 2018',<EOF>`	`'Yanxi Palace - 2018',<EOF>`	✓
✓	`"'abc?svn'"`	`'abc?svn',<EOF>`	`'abc?svn',<EOF>`	✓
✓	`"'1.0e-12'"`	`'1.0e-12',<EOF>`	`'1.0e-12',<EOF>`	✓
✓	`"'0.001''"`	`'0.001',Error Token '`	`'0.001',Error Token '`	✓
✓	`"'0.001'''"`	`'0.001''',<EOF>`	`'0.001''',<EOF>`	✓

Câu hỏi **4**

Đúng

Đạt điểm 1,00 trên 1,00

Use ANTLR to write regular expressions describing PHP's integers (in decimal) which is a sequence of digits (0-9) starting with a non-zero digit or only a zero. Integer literals may contain underscores (_) between digits, for better readability of literals but these underscores are removed by PHP's scanner.

For example:

Test	Result
1_234_567	1234567,<EOF>

Answer: (penalty regime: 10, 20, ... %)

Ace editor not ready. Perhaps reload page?
Falling back to raw text area.

HLang.interp

HLang.tokens

HLangLexer.interp

HLangLexer.py

HLangLexer.tokens

HLangParser.py

HLangVisitor.py

	Test	Expected	Got	
✓	1_234_567	1234567,<EOF>	1234567,<EOF>	✓
✓	1234	1234,<EOF>	1234,<EOF>	✓
✓	123_4	1234,<EOF>	1234,<EOF>	✓
✓	76890_1	768901,<EOF>	768901,<EOF>	✓
✓	2.01	2,Error Token .	2,Error Token .	✓

Câu hỏi **5**

Đúng

Đạt điểm 1,00 trên 1,00

Use ANTLR to write regular expressions describing a valid IPv4 address. It consists of exact 4 strings, whose length is from 1 to 3, of digits (0-9) but not starting with 0 unless the string is 0. The strings are separated by one dot (.).

For example:

Test	Result
`192.168.0.1`	`192.168.0.1,<EOF>`

Answer: (penalty regime: 10, 20, ... %)

Ace editor not ready. Perhaps reload page?
Falling back to raw text area.

HLang.interp

HLang.tokens

HLangLexer.interp

HLangLexer.py

HLangLexer.tokens

HLangParser.py

HLangVisitor.py

	Test	Expected	Got	
✓	`192.168.0.1`	`192.168.0.1,<EOF>`	`192.168.0.1,<EOF>`	✓
✓	`192.168.01.1`	`Error Token 1`	`Error Token 1`	✓
✓	`152.101.227.0`	`152.101.227.0,<EOF>`	`152.101.227.0,<EOF>`	✓
✓	`1168.93.28.1`	`Error Token 1`	`Error Token 1`	✓
✓	`2.1687.0.2`	`Error Token 2`	`Error Token 2`	✓

Câu hỏi **6**

Đúng

Đạt điểm 1,00 trên 1,00

Use **ANTLR** to write a regular expression for the token **SHEXA** that describes hexadecimal number strings satisfying all the following requirements:

- Non-empty
- Corresponds to an even integer
- The first character must be a digit
- Case-insensitive (does not distinguish between lowercase and uppercase letters)
- Do not use actions when writing the regular expression for **SHEXA**

Examples of valid strings for **SHEXA**: `12`, `21A`, `3dC`, `2`

Examples of invalid strings for **SHEXA**: `A12` (first character is a letter), `1B` (corresponds to 27, which is not an even integer).

For example:

Test	Result
12	12,<EOF>

Answer: (penalty regime: 10, 20, ... %)

Ace editor not ready. Perhaps reload page?
Falling back to raw text area.

HLang.interp

HLang.tokens

HLangLexer.interp

HLangLexer.py

HLangLexer.tokens

HLangParser.py

HLangVisitor.py

Câu hỏi **7**

Đúng

Đạt điểm 1,00 trên 1,00

When enrolling at the University of Technology, students are required to set an account name called **BKNetID**, which consists of three components in this order: given name, family name, and a user-chosen string. A dot (.) must be placed between the given name and the family name. The given name and family name are strings containing only lowercase letters, with a minimum length of 1. The user-chosen string is 1 to 5 characters long and may include lowercase letters, digits, dots, and underscores, but it must not end with a dot.

Examples: `duy.tran2903`, `duy.tran.3_12` are valid BKNetIDs, whereas `duy.tran2903.` or `duy2.tran2903` are invalid.

Use **ANTLR** to write the regular expression for the BKNetID described above. Students must use a **fragment** to receive full credit.

For example:

Test	Result
`duy.tran2903`	`duy.tran2903,<EOF>`

Answer: (penalty regime: 10, 20, ... %)

Ace editor not ready. Perhaps reload page?
Falling back to raw text area.

HLang.interp

HLang.tokens

HLangLexer.interp

HLangLexer.py

HLangLexer.tokens

HLangParser.py

HLangVisitor.py

	Test	Expected	Got	
✓	`duy.tran2903`	`duy.tran2903,<EOF>`	`duy.tran2903,<EOF>`	✓